E-Commerce Product Entity Resolution

Using Data Structures and Algorithms (DSA)

PROJECT DOCUMENTATION

Frontend: React.js **Backend:** Node.js + Express.js
DSA: HashMap, HashSet, ArrayList, String Matching, Searching & Sorting

1. Abstract

The E-Commerce Product Entity Resolution & Matching Platform identifies and groups identical products listed by different online sellers. It normalizes product names and compares brand, model, storage, color and other specifications. HashMap is used for fast product grouping, HashSet for duplicate detection, ArrayList for seller listings, string matching for comparison, and sorting for price ranking. React.js provides the frontend while Node.js and Express.js provide the backend. The system displays matched products and highlights the lowest seller price. The project demonstrates a practical application of core DSA concepts without requiring machine learning.

2. Introduction

Online marketplaces contain multiple listings for the same real-world product. Sellers may use different names, spaces, capitalization and attribute formats. Entity resolution determines whether two records represent the same product. This project solves that problem with normalization, matching, hashing, searching and sorting.

3. Problem Statement

Different sellers can describe the same product differently, causing duplicate or fragmented listings. The system must efficiently normalize, compare, group and search these records and provide useful price comparison.

4. Objectives

Identify identical products across sellers; normalize product information; detect duplicates; group matched listings; search products; sort seller prices; and demonstrate practical DSA usage.

5. Software Requirements

Frontend: React.js, JSX, CSS. Backend: Node.js and Express.js. Programming: JavaScript. DSA: HashMap, HashSet, ArrayList, String Matching, Searching and Sorting. Tools: VS Code, npm, Git/GitHub. Browser: Chrome/Edge/Firefox.

6. Hardware Requirements

Processor: Intel Core i3 or equivalent. RAM: minimum 4 GB, 8 GB recommended. Storage: at least 500 MB free. Display: 1366×768 or higher. Internet is required for cloud deployment.

7. Literature Survey

Entity-resolution research commonly uses preprocessing, indexing, candidate generation, record comparison and duplicate detection. Data-matching literature provides a general workflow for resolving records, while product-mapping research shows that heterogeneous product titles and attributes make cross-seller matching challenging. This project applies these ideas in a simple DSA implementation.

8. System Architecture

React.js Frontend → REST API → Node.js + Express → DSA Matching Engine → Product Groups → Search & Price Sorting → Website Output.

9. Methodology

1) Collect seller records. 2) Normalize text. 3) Extract brand/model/storage/color. 4) Generate a matching key. 5) Group records using HashMap. 6) Detect duplicates with HashSet. 7) Compare names with string matching. 8) Sort prices. 9) Return results to React.

10. DSA Used

DSA	Purpose
HashMap	Fast product grouping and lookup
HashSet	Duplicate detection
ArrayList	Stores seller listings
String Matching	Compares normalized product names
Searching	Finds requested products
Sorting	Ranks prices and finds best deal

11. Product Dataset

Product	Seller	Price
iPhone 15 128GB Black	Amazon	■69,999
iPhone 15 128GB Black	Flipkart	■68,999
iPhone 15 128GB Black	Croma	■69,500
Galaxy S24 256GB	Amazon	■74,999
Galaxy S24 256GB	Flipkart	■72,499
Dell Inspiron 15	Amazon	■54,990
Dell Inspiron 15	Flipkart	■53,490
Sony WH-1000XM5	Amazon	■29,990
Sony WH-1000XM5	Flipkart	■28,499

12. Website Output – Product Matching

PRODUCT MATCH	STATUS
Apple iPhone 15 128GB Black	✓ MATCHED
Amazon	■69,999
Flipkart	■68,999 ★ BEST PRICE
Croma	■69,500

Website Features

Search box, product cards, matched-product badge, seller price comparison, best-price indicator, and DSA information. The React interface sends product requests to the Node.js backend through REST APIs.

13. Sample Output

Search: iPhone 15
Result: Same product found across Amazon, Flipkart and Croma.
Best Price: Flipkart – ■68,999

Backend output:
Matched Product: Apple iPhone 15 128GB Black
Flipkart ■68999
Croma ■69500

Amazon ■69999
Best Price: Flipkart ■68999

14. Testing

Test Case	Expected Result	Status
Search iPhone 15	Matched group displayed	PASS
Duplicate seller record	Duplicate ignored	PASS
Different spacing in name	Same group detected	PASS
Price comparison	Lowest price identified	PASS
Unknown product	No matching product	PASS

15. Advantages

Easy DSA implementation; fast lookup; duplicate reduction; price comparison; modern React UI; and clear demonstration of hashing, searching, sorting and string matching.

16. Limitations

The sample project uses a limited dataset and basic string matching. Production use would need a persistent database, stronger fuzzy matching and real-time seller data.

17. Future Scope

Add MySQL/MongoDB, fuzzy matching or edit distance, authentication, product images, real-time e-commerce APIs, recommendations and machine-learning-based matching.

18. Conclusion

The project demonstrates how core Data Structures and Algorithms can solve a practical e-commerce problem. HashMap, HashSet, ArrayList, string matching, searching and sorting work together to identify duplicate products, group listings and find the best price. React.js and Node.js provide a practical web interface around the DSA engine.

19. References

Peter Christen, Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection, Springer, 2012.
Brizan & Tansel, survey literature on entity resolution and record linkage methodologies, 2006.
Köpcke et al., research on frameworks for entity matching.
React.js, Node.js and Express.js documentation.